

计算机系统基础

Lab1 实验报告

DataLab

姓名： 吴向杰

学号： 25300190057

日期： 2026 年 9 月 28 日

目录

1	实验目的	1
2	实验环境与编码规则	1
3	实验过程	1
4	各函数的实现思路	3
5	测试结果	8
6	实验小结	9
7	参考资料	9

1 实验目的

本实验通过完成 `bits.c` 中的 19 个函数，练习整数补码、掩码、移位以及 IEEE 754 单精度浮点数的位级操作。除了计算结果正确，每个函数还需要满足合法运算符、常量范围和最大操作数的限制。实验的重点是理解这些限制下的实现方法，并正确处理符号变化、整数溢出和浮点舍入等边界情况。

2 实验环境与编码规则

- 操作系统：Windows，编译和测试在 WSL 2 的 Ubuntu 中完成；
- 编译环境：x86-64 Linux，支持 32 位程序编译；
- 实验目录：Windows 下为 `E:\ICS \Lab 1`，WSL 中为 `/mnt/e/ICS/Lab1`；
- 工具：GCC、Make、Python 3，以及仓库提供的 `dlc` 和 `btest`；
- 编译参数：`-O -Wall -fwrapv -m32`；
- 个人仓库：<https://github.com/wuxiangjie84/ICS-Lab1>。

整数题采用实验规定的 32 位补码模型，有符号右移为算术右移，运算超出 32 位时保留低 32 位，移位量必须在 0 到 31 之间。一般整数题只允许使用 `! ~ & ^ | + << >>` 和 0 到 255 的整数常量，其中 P2 只能使用 `~` 和 `&`。浮点题允许条件和循环语句，但仍然通过整数操作处理编码，不直接使用浮点类型、浮点运算或类型转换。

下文中的较大十六进制常量用于说明目标位模式；整数题代码中的对应掩码均由合法的小常量构造。记 32 位有符号整数的最小值为 $\text{INT_MIN} = -2^{31}$ ，最大值为 $\text{INT_MAX} = 2^{31} - 1$ 。

3 实验过程

实验使用课程提供的 DataLab 模板，仅填写 P1 至 P19 的函数体，保留函数签名、测试程序和评分配置。代码修改后，先清除旧的编译产物，再重新编译并运行两类检查：

```
make clean
make all
./check_ops.py bits.c
./btest
```

其中，`check_ops.py` 检查合法运算符和操作数数量，`btest` 检查输出结果。两项检查需要同时通过。调试单个函数时，可以使用 `./btest -f 函数名`；完整流程也可通过 `./test.sh`

执行。测试时重点关注最小负整数、移位边界、舍入中点和浮点特殊值。

4 各函数的实现思路

4.1 P1: signMask

目标是得到只有最高位为 1 的掩码。32 位整数从最低位开始编号为 0 到 31，因此把常量 1 左移 31 位即可得到 0x80000000。这里只使用一次左移，且常量 1 和 31 都在允许范围内。

4.2 P2: bitXor

异或在两个对应位不同时为 1，可以写成“至少有一个位为 1，并且不是两个位都为 1”。根据德摩根定律，按位或可以由取反和按位与表示，因此实现为：

```
~(~x & ~y) & ~(x & y)
```

前半部分相当于 $x \mid y$ ，后半部分排除 $x \& y$ 为 1 的位置。整个表达式只使用题目允许的 $\sim$ 和 $\&$ 。

4.3 P3: negativePart

算术右移 $x \gg 31$ 会在负数时得到全 1，在非负数时得到全 0。补码模型下， $\sim x + 1$ 表示取相反数，将它与这个符号掩码相与，就能在负数时保留结果，在其余情况下返回 0。对于 INT_MIN，其正值无法用有符号 32 位整数表示，按照本实验的位向量规则，结果仍保留为 0x80000000。

4.4 P4: copyByteWithin

一个字节占 8 位，所以源字节和目标字节的偏移量分别为 $\text{src} \ll 3$ 和 $\text{dst} \ll 3$ 。先右移并与 0xFF 相与，提取源字节；再将 0xFF 左移到目标位置，得到目标字节掩码。原数与掩码的取反相与后，目标字节被清零，其余字节不变，最后用按位或写入源字节。例如，将 0x11223344 的第 0 字节复制到第 2 字节，结果为 0x11443344。

4.5 P5: logicalShift

直接执行 $x \gg n$ 时，负数的高位会补 1，因此还需要清除这些补入的位。使用的保留掩码为：

```
~(((1 << 31) >> n) << 1)
```

当 $n > 0$ 时，这个掩码的低 $32 - n$ 位为 1，高 n 位为 0；当 $n = 0$ 时掩码为全 1。将算术右移结果与掩码相与即可得到逻辑右移结果。这种构造在 $n = 0$ 和 $n = 31$ 时都不需要执行移位 32 位的操作。

4.6 P6: swapNibblePairs

先由常量 0x0F 构造 0x0F0F0F0F，它选中每个字节的低 4 位。 $(x \& \text{mask}) \ll 4$ 把低 4 位移到高 4 位； $(x \gg 4) \& \text{mask}$ 提取原来的高 4 位，并放到低 4 位。两部分按位或后完成交换。右移后的掩码同时清除了负数右移时补入的高位 1。

4.7 P7: secondLowestZeroBit

设 $\text{filled} = x \mid (x + 1)$ 。加 1 会将最低的一个 0 变为 1，并使其右侧连续的 1 变为 0；与原数按位或后，这些低位的 1 又被保留，因此 filled 恰好把原数最低的一个 0 填成了 1。

接着计算 $\sim \text{filled} \& (\text{filled} + 1)$ ，即可单独保留 filled 中最低的 0 所在的位置，也就是原数第二个 0 的位置。如果原数不足两个 0， filled 为全 1，加 1 后低 32 位为 0，最终也返回 0。

4.8 P8: oddParity

异或可以记录 1 的个数是奇数还是偶数。依次将当前结果与其右移 16、8、4、2、1 位后的值异或，就能把全部 32 位的奇偶性汇总到最低位。最低位为 1 表示原数有奇数个 1，为 0 表示有偶数个 1。本题要求返回补成奇校验所需的位，因此最后使用 $!(x \& 1)$ 。例如，5 的二进制为 101，有两个 1，返回值为 1。

4.9 P9: rotateRightBits

循环右移需要把移出最低端的位放回最高端。先用 $n \& 31$ 得到实际移位量 s ，再用 P5 的掩码方法得到逻辑右移 s 位的部分。另一部分由原数左移 $(32 - s) \bmod 32$ 位得到，两部分按位或即为结果。代码中用 $(\sim \text{shift} + 1) \& 31$ 计算左移量，所以 $s = 0$ 时左右移位量均为 0，结果保持不变，不会出现移位 32 位的情况。

4.10 P10: roundEvenPow2

设 $B = 2^n$ 是舍入步长, $q = \lfloor x/B \rfloor$ 是商, $r = x - qB$ 是余数。当 $r < B/2$ 时向下取整, $r > B/2$ 时向上取整; 恰好等于 $B/2$ 时选择偶数商。实现时先加入偏置:

$$\text{bias} = B/2 - 1 + (q \& 1),$$

再清除结果的低 n 位。商为偶数时, 偏置为 $B/2 - 1$, 恰好处于中点也不会进位; 商为奇数时, 偏置为 $B/2$, 中点会进位到相邻的偶数商。例如, 10 舍入到 4 的倍数得到 8, 而 14 舍入到 4 的倍数得到 16。代码用 $+ \sim 0$ 表示减 1, 避免使用整数题禁止的减号。

4.11 P11: midpointTowardFirst

直接计算 $x + y$ 可能溢出, 因此先计算向下取整的中点:

$$L = (x \& y) + ((x \oplus y) \gg 1).$$

这里 $\oplus$ 表示按位异或, $\gg$ 表示算术右移。其依据是数学上的恒等式 $x + y = 2(x \& y) + (x \oplus y)$, 这种拆分避免了先保存可能溢出的完整和。

若和为奇数, 还需要决定取 L 还是 $L + 1$ 。当 $x > y$ 时, 第一个参数位于中点右侧, 应取 $L + 1$; 其余情况取 L 。比较时先区分两数是否异号: 异号时由符号判断, 同号时再使用差的符号, 避免用溢出的差作判断。因此 `midpointTowardFirst(4,7)` 返回 5, 而交换参数后返回 6。

4.12 P12: isBetweenEitherOrder

分别构造 $x < a$ 和 $x < b$ 的判断结果。如果两个结果不同, 说明 x 位于两个端点之间, 但可能排除了一个端点, 因此还需要加上 $x = a$ 或 $x = b$ 的情况。等号判断用 $!(x \wedge a)$ 和 $!(x \wedge b)$ 实现。

比较大小时仍先检查符号是否不同。异号时直接采用 x 的符号, 同号时采用 $x - a$ 或 $x - b$ 的符号。这样即使端点顺序相反、跨越正负区间, 或两个端点相等, 也能得到包含端点的区间判断结果。

4.13 P13: mul5Sat

通过相加依次得到 $2x$ 、 $4x$ 和 $5x$ 的低 32 位结果。若乘 5 的精确结果没有溢出, 这些中间结果会保持原来的符号; 如果发生溢出, 至少一次相加会产生符号翻转。将相邻结果的异或按位或起来, 再右移 31 位, 就得到是否饱和的全 0 或全 1 掩码。

发生溢出时, 正数应返回 `INT_MAX`, 负数应返回 `INT_MIN`。代码用 $(1 \ll 31) \wedge \sim(x \gg 31)$ 构造相应的边界值, 最后由溢出掩码选择边界值或正常的 $5x$ 结果。

4.14 P14: classifyAdd3

先计算 $s = x + y$ 的低 32 位，再计算 $t = s + z$ 的低 32 位。每一步都依据“同号数相加，结果符号相反”判断有符号溢出。对于第一步， $((x \wedge \text{firstSum}) \& (y \wedge \text{firstSum})) \gg 31$ 在溢出时为全 1，否则为 0；第二步使用相同方法。

设每一步的溢出修正值为 c_1, c_2 ：正溢出记为 1，负溢出记为 -1，未溢出记为 0。那么三个数的精确数学和满足

$$S = t + 2^{32}(c_1 + c_2).$$

连续两次加法不会发生同方向的有符号溢出，所以 $c_1 + c_2$ 只可能是 -1、0 或 1，正好对应题目所要求的三个返回值。这也能处理溢出相互抵消的情况。例如 $\text{INT_MAX} + \text{INT_MAX} + \text{INT_MIN} = \text{INT_MAX} - 1$ ，虽然中间发生溢出，最终仍应返回 0。

4.15 浮点题使用的表示方法

P15 至 P17 都以整数保存单精度浮点数的编码。记符号位为 s ，8 位阶码的无符号值为 E ，23 位小数字段的无符号值为 F ，则有限数的值为

$$f = \begin{cases} (-1)^s F 2^{-149}, & E = 0, \\ (-1)^s (2^{23} + F) 2^{E-150}, & 1 \leq E \leq 254. \end{cases}$$

$E = 0, F = 0$ 对应正零或负零； $E = 255, F = 0$ 对应无穷， $E = 255, F \neq 0$ 对应 NaN。需要丢弃低位时，先比较被丢弃部分与一半的大小：超过一半则进位，恰好一半时仅在保留部分的最低位为 1 时进位，从而实现最近偶数舍入。

4.16 P15: floatScaleThreeHalves

首先拆出符号、阶码和小数字段。阶码为 255 时直接返回原编码，因此无穷和 NaN 都得到保留。对于非规格化数，先计算 $P = 3F$ ，再将 $P/2$ 舍入到整数。此时 $(\text{product} \& 3) == 3$ 表示被丢弃的最低位为 1，且保留部分最低位也为 1，正是中点需要进位的情况。舍入结果若进入第 23 位，会自然形成最小阶码范围内的规格化数编码。

对于规格化数，补上隐含的最高位，得到整数有效数 $M = 2^{23} + F$ ，再计算 $P = 3M$ 。如果 $P < 2^{25}$ ，将其右移 1 位；否则右移 2 位并把阶码加 1，使保留的有效数维持在 24 位范围。随后根据余数执行最近偶数舍入，若舍入产生新的最高位进位，再调整有效数和阶码。阶码达到 255 时返回同号无穷。整个过程保留原符号位，所以正负零的符号也不会丢失。

4.17 P16: floatRoundEven

先按阶码划分情况。 $E \geq 150$ 的有限数已经没有小数部分，连同阶码为 255 的 NaN 和无穷一起直接返回原编码。 $E < 126$ 时绝对值小于 0.5，返回带原符号的零。 $E = 126$ 时数值的绝对值位于 $[0.5, 1)$ ：小数字段为零表示恰好 0.5，舍入到零；其余情况舍入到同号的 1。

对于 $127 \leq E < 150$ ，小数部分对应编码中的低 $k = 150 - E$ 位。先用掩码清除这 k 位，再根据被清除部分是否超过一半、保留整数是否为奇数决定是否进位。输出仍然是浮点编码，而不是普通的整数。例如，1.5 舍入为 2.0，2.5 舍入也为 2.0；负数舍入为零时保留负零。

4.18 P17: float_i2f

零直接返回零编码。非零输入先记录符号，并在无符号变量中计算幅值，避免对 INT_MIN 直接进行有符号取反。随后从第 31 位向下寻找最高的 1，设其位置为 p ，对应阶码为 $p + 127$ 。

当 $p \leq 23$ 时，有效位不超过 24 位，可以通过左移精确表示；当 $p > 23$ 时，保留最高 24 位，并将丢弃的 $p - 23$ 位用于最近偶数舍入。如果舍入导致有效数由 24 位变为 25 位，则再右移一位并增加阶码。例如，整数 16777217 位于两个相邻单精度数之间，按中点取偶规则转换为 16777216。

4.19 P18: bitCount

采用分组累加的方法统计 1 的数量。先用 0x55555555 将相邻两位分开，计算每两位中 1 的个数；再用 0x33333333 将相邻的两组相加，得到每四位的计数；接着用 0x0F0F0F0F 得到每个字节的计数。最后通过右移 8 位和 16 位继续累加，将总数集中到低位。计数范围为 0 到 32，低 6 位足以保存结果，因此返回 $x \& 63$ 。

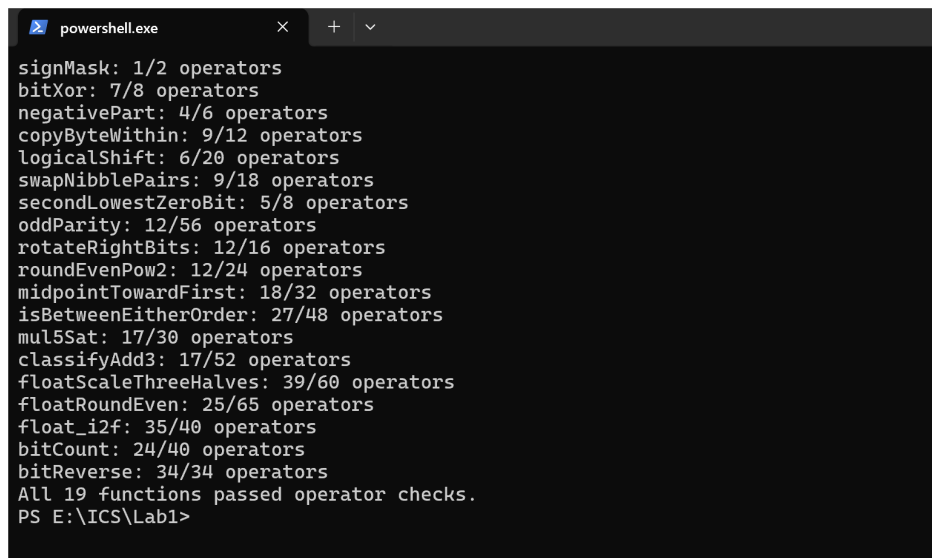
4.20 P19: bitReverse

先交换相邻的单个位，再交换相邻的两位组，最后交换每个字节内的两个四位组。这三步完成各字节内部的位逆序，再把四个字节按相反顺序拼接，就得到整个 32 位整数的逆序结果。

本题的操作数上限较紧，因此掩码不分别从头构造。先生成 $\text{mask4} = 0x0F0F0F0F$ ，再利用 $\text{mask4} \wedge (\text{mask4} \ll 2)$ 得到 0x33333333，继续用 $\text{mask2} \wedge (\text{mask2} \ll 1)$ 得到 0x55555555。最后直接重排四个字节，使总操作数为 34，满足上限。例如，0x80000004 逆序后得到 0x20000001。

5 测试结果

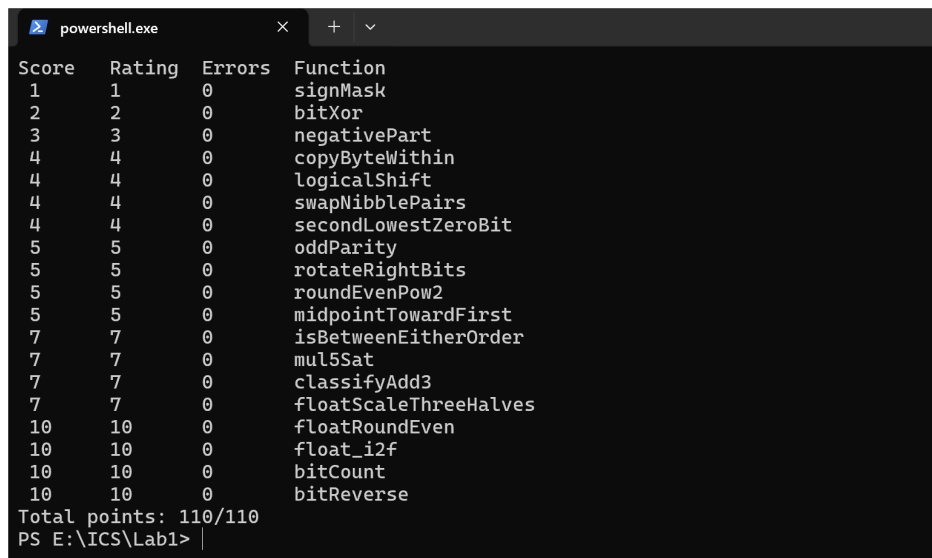
`./check_ops.py bits.c` 的结果如图 1 所示，19 个函数均通过规则检查，操作数未超过各题限制。



```
powershell.exe
signMask: 1/2 operators
bitXor: 7/8 operators
negativePart: 4/6 operators
copyByteWithin: 9/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 9/18 operators
secondLowestZeroBit: 5/8 operators
oddParity: 12/56 operators
rotateRightBits: 12/16 operators
roundEvenPow2: 12/24 operators
midpointTowardFirst: 18/32 operators
isBetweenEitherOrder: 27/48 operators
mul5Sat: 17/30 operators
classifyAdd3: 17/52 operators
floatScaleThreeHalves: 39/60 operators
floatRoundEven: 25/65 operators
float_i2f: 35/40 operators
bitCount: 24/40 operators
bitReverse: 34/34 operators
All 19 functions passed operator checks.
PS E:\ICS\Lab1>
```

图 1: 运算符与操作数检查结果

`./btest` 的结果如图 2 所示，所有函数的 Errors 均为 0，总分为 110/110。这里记录的是 WSL Ubuntu 中的本地测试结果。



```
powershell.exe
Score  Rating  Errors  Function
-----
1      1        0      signMask
2      2        0      bitXor
3      3        0      negativePart
4      4        0      copyByteWithin
4      4        0      logicalShift
4      4        0      swapNibblePairs
4      4        0      secondLowestZeroBit
5      5        0      oddParity
5      5        0      rotateRightBits
5      5        0      roundEvenPow2
5      5        0      midpointTowardFirst
7      7        0      isBetweenEitherOrder
7      7        0      mul5Sat
7      7        0      classifyAdd3
7      7        0      floatScaleThreeHalves
10     10       0      floatRoundEven
10     10       0      float_i2f
10     10       0      bitCount
10     10       0      bitReverse
Total points: 110/110
PS E:\ICS\Lab1>
```

图 2: 函数正确性测试结果

6 实验小结

本实验中的整数题主要依靠掩码实现选择和分组处理，比较运算则需要区分同号与异号情况。浮点题的关键在于正确理解阶码和有效数，并在丢弃低位时完整处理舍入。规则检查与正确性检查分别验证不同内容，代码需要同时满足这两方面要求。

7 参考资料

- [1] 复旦大学《计算机系统基础》课程：Lab1 DataLab 实验说明。用于核对实验流程、编码要求和报告要求。

<https://ics-26fall-fdu.github.io/labs/lab1-data-lab/>

- [2] 课程 DataLab 模板仓库：README.md。用于核对 32 位机器模型、逐题限制与测试方法。

<https://github.com/ICS-26Fall-FDU/DataLab/blob/main/README.md>

- [3] 课程 DataLab 模板仓库：bits.c、tests.c。用于核对函数语义、输入边界、特殊值和参考行为。

<https://github.com/ICS-26Fall-FDU/DataLab>